

HW4: CUDA Matmul Optimization

Release: 2026-05-08 · **Deadline:** 2026-05-22 (Fri) 23:59

1. Introduction

Matrix multiplication is the computational workhorse of modern HPC and deep learning. In this assignment you will implement an optimized single-GPU, single-precision GEMM (SGEMM) kernel in CUDA and compete against NVIDIA's hand-tuned cuBLAS library.

The operation is (row-major):

$$C = \alpha * (A @ B) + \beta * C$$

where A is $M \times K$, B is $K \times N$, C is $M \times N$. For this homework $M = N = K$ and is swept over $\{128, 256, 512, 1024, 2048, 4096\}$.

Performance is reported in GFLOPS:

$$\text{GFLOPS} = 2 * M * N * K * \text{repeat_times} / \text{elapsed_time} / 1e9$$

The test harness (`main.cu`) runs one warm-up iteration plus 50 timed iterations per size, using `cudaEvent` for timing.

2. Requirements

1. Implement a custom CUDA kernel in `kernels/student_kernel.cu` that computes the GEMM above.
2. Your kernel must pass correctness verification against the cuBLAS reference (tolerance $1e-2$) for **every** test size.
3. Write a short report analysing the optimizations you applied and their measured impact.

Forbidden

- Calling cuBLAS / cuDNN / CUTLASS / any vendor GEMM library from your kernel.
- Modifying any file outside `kernels/student_kernel.cu`. Grading uses the official skeleton; your other edits will be discarded and may break the build.

3. Environment

GPU	NVIDIA Tesla V100 (Compute Capability 7.0, <code>sm_70</code>)
CUDA	loaded via <code>module load cuda</code>
Scheduler	Slurm (<code>srun</code> / <code>sbatch</code>)

Project ID **ACD115083**

A typical interactive resource request:

```
srun -N 1 -n 1 --gpus-per-node 1 -A ACD115083 -t 1 ./main
```

Useful **srun** flags:

flag	meaning
-A	project ID (ACD115083)
-N	number of nodes
-n	number of processes
-c	CPUs per process
-t	time limit (minutes)
-J	job name
--gpus-per-node	GPUs per node

4. Execution

Build

```
module load cuda
make                               # default CUDA_ARCH=70 (V100)
```

Run

```
srun -N 1 -n 1 --gpus-per-node 1 -A ACD115083 -t 1 ./main
```

Expected stdout

```
Using device: Tesla V100-SXM2-32GB
Max size: 4096
Selected kernel: 1
Running size: 128...   avg time: 0.000xxxs, performance:   xxx.xxx GFLOPS
Running size: 256...   avg time: 0.000xxxs, performance:  xxxx.xxx GFLOPS
...
Running size: 4096...   avg time: 0.0xxxxxs, performance: xxxxx.xxx GFLOPS
```

Grading parses this stdout. Do not change the print format.

5. Submission

- **Deadline:** 2026-05-22 (Fri) 23:59
- **Filename:** `Team_<Team Number>_HW4.zip`
- **Contents** (at zip root):
 1. `kernels/student_kernel.cu` — your implementation
 2. `Team_<Team Number>_HW4_report.pdf` — see §7

TA will drop your `student_kernel.cu` into the official skeleton, rebuild with the commands in §4, and run the grading harness. Any submission that fails to compile with the official skeleton will lose points.

6. Grading

Component	Weight	Notes
Correctness	10%	All 6 sizes must pass verification (tol = $1e-2$).
Performance Tier (size = 4096)	40%	Cumulative, see tier table.
Competition Rank (size = 4096 GFLOPS)	20%	$score = 20 \times (N + 1 - rank) / N$, where N = number of teams.
Report	30%	See §7.

Performance Tiers

Tiers are **absolute GFLOPS thresholds** at `size = 4096`, measured once by the TA on the grading V100. Thresholds are cumulative — reaching T3 awards the full 30%, reaching T5 awards the full 40%. The % of `cuBLAS` column is informative only; grading uses the absolute GFLOPS number.

Tier	Threshold (GFLOPS @ 4096)	≈ % of cuBLAS	Cumulative score
T1	≥ 200	1.5 %	+20%
T2	≥ 2050	15 %	+25%
T3	≥ 3700	27 %	+30%
T4	≥ 7600	55 %	+37%
T5	≥ 9500	68 %	+40%

Failure modes

- If correctness fails on **any** size, both the Performance-Tier and Competition-Rank components become 0.
- Late submissions are multiplied by **70%**.
- Submissions that call a forbidden library (`cuBLAS` / `cuDNN` / `CUTLASS`) receive 0 on correctness, tier, and rank.

7. Report

Keep it short. Suggested sections:

- **Optimizations applied**, in the order you added them, with the measured GFLOPS after each step. A small table is enough.
- **Final kernel design** — key choices: tile sizes, thread tile, memory layout, vectorization, synchronization strategy. A short paragraph or a diagram.
- **What you tried that didn't work**, if anything. Brief.
- *(Optional)* A scaling plot of GFLOPS vs. matrix size.

Length guideline: 2 pages is plenty; quality over length.

8. Hints

In rough order of difficulty, the standard optimization directions for SGEMM on NVIDIA GPUs are:

1. **Global-memory coalescing** — re-index threads so that a warp's reads/writes are contiguous.
2. **Shared-memory tiling** — stage tiles of A and B into shared memory and reuse them across all threads in the block.
3. **1D thread blocktiling** — each thread computes multiple output elements, increasing arithmetic intensity.
4. **2D thread blocktiling + register caching** — each thread computes an $m \times n$ micro-tile held in registers.
5. **Bank-conflict-free shared memory** — e.g. transpose A's shared tile so that inner-product accesses stride across banks without conflicts.
6. **Vectorized loads/stores** — issue 128-bit transactions with `float4`.
7. **Warp-level tiling / autotuning** — partition the block tile across warps and sweep parameters.

9. Policy

Fairness. TA strictly uses the commands in §4 together with the official skeleton. No custom flags or environment tweaks will be applied.

Academic integrity. Do not copy code from public repositories or from other teams. Discussing high-level ideas and optimization directions is fine; sharing code is not.